An FPGA Implementation of the LMS Adaptive Filter for Audio Processing

Ahmed Elhossini, Shawki Areibi, Robert Dony
School of Engineering
University of Guelph
Guelph , Canada , N1G 2W1
Email: {aelhossi,sareibi,rdony}@uoguelph.ca

Abstract

This paper proposes three different architectures for implementing a least mean square (LMS) adaptive filtering algorithm, using a 16 bit fixed-point arithmetic representation. These architectures are implemented using the Xilinx multimedia board as an audio processing system. The on-board AC97 audio codec is used for audio capture/playback, and the Virtex-II FPGA chip is used to implement the three architectures. A comparison is then made between the three alternative architectures with different filter lengths for performance and area. Results obtained show an improvement by 90% in the critical part of the algorithm when a hardware accelerator is used to perform it over a pure software implementation. This results in a total speed up of $3.86\times$*. However, using a pure hardware implementation results in a much higher performance with somewhat lower flexibility. It shows a speed up close to* $82.6\times$ *over the software implementation.*

1 Introduction

In the last few decades the demand for portable and embedded digital signal processing (DSP) systems has increased dramatically. Applications such as cell phones, hearing aids, and digital audio devices are applications with stringent constraints such as area, speed and power consumption. These applications require an implementation that meet these constraints with the shortest time to market. The possible alternative implementations that can be used range from an ASIC custom chip, general purpose processor (GPP) to DSP processors. While the first choice could provide the solution that meets all the hard constraints, it lacks the flexibility that exist in the other two, and also its design cycle is much longer. Reconfigurable computing is gaining much attention as a prototyping and implementation technology of digital systems. Using programmable deceives (like FPGAs) for DSP applications could narrow the gap between the flexibility of GPP, and programmable DSP processors, and the high performance of dedicated hardware using ASIC technology [7].

Modern FPGAs contain many resources that support DSP applications such as embedded multipliers, multiply accumulate units (MAC), and processor cores. These resources are implemented in the FPGA fabric and optimized for high performance and low power consumption. Also many soft cores are available from different vendors that provide a support for the basic blocks in many DSP applications [3, 7, 5].

The availability of hard/soft core processors in modern FPGAs allow moving DSP algorithms written for GPP or DSP processors to FPGAs using the core processors. An alternative approach is to move part of the algorithm into hardware (HW) to improve performance. This is a form of HW/SW Co-design, that requires profiling the software to efficiently partition it between HW and SW. This solution could result in a more efficient implementation as part of the algorithm is accelerated using HW while the flexibility is maintained. A third, usually more efficient, and more complex alternative is to convert the complete algorithm into hardware [9]. Although this solution is attractive in terms of performance, area, and power consumption, the design cycle is much longer and more complex.

In this work, the LMS adaptive algorithm [12] is implemented by three different architectures on an FPGA. The algorithm is used to process a speech signal to enhance its signal to noise ratio (SNR). The Xilinx Multimedia board is used to implement the architectures. The on-board audio codec (AC97) is used for audio capture/playback and the Xilinx Virtex-II FPGA chip is used to realize the three implementations. A pure software architecture of the algorithm is first proposed using MicroBlaze (MB) soft-core RISC processor. An FIR filter core is then proposed to implement a HW/SW Co-design architecture with the existing

MB. Finally a pure HW architecture is mapped and tested. The performance and area of each architecture is compared for different adaptive filter lengths.

The remainder of this paper is organized as follows: Section 2 gives necessary background on the LMS algorithm, the multimedia board and tools used for implementation. Section 3 introduces detailed implementation of each architecture. Section 4 presents the implementation results, and finally section 5 concludes the paper.

2 Background

The LMS algorithm is a widely used technique for adaptive filtering. Its origin is attributed to Windrow and Hoff (1960) [12, 11, 8]. It is based on the estimation of the gradient toward the optimal solution using the statistical properties of the input signal. A significant feature of the LMS algorithm is simplicity. In this algorithm filter weights are updated with each new sample as required to meet the desired output. The computation required for weights update is illustrated by equation (1). If the input values $u(n), u(n-1), u(n-2)....u(n-N+1)$ form the tap input vector **u(n)**, where N denotes the filter length, and the weights $\widehat{w}_0(n), \widehat{w}_1(n).....\widehat{w}_{N-1}(n)$ form the tap weight vector $\widehat{w}(n)$ at iteration n, then the LMS algorithm is given by the following equations:

$$\begin{aligned} y(n) &= \widehat{w}^H(n)u(n) \\ e(n) &= d(n) - y(n) \\ \widehat{w}(n+1) &= \widehat{w}(n) + \mu u(n)e(n) \end{aligned} \quad (1)$$

In equation (1), $y(n)$ denotes the filter output, $d(n)$ denotes the desired output, $e(n)$ denotes the filter error (the difference between the desired filter output and current filter output) which is used to update the TAP weights, μ denotes a learning rate, and $\widehat{w}(n+1)$ denotes the new weight vector that will be used by the next iteration.

In [10] the LMS algorithm is used as a noise canceller on the Xilinx Spartan2E FPGA. The implementation is based on a MAC unit that is used to multiply-accumulate the filter output and weights update. Distributed arithmetic is used to implement the LMS algorithm on an Altera Stratix FPGA [6]. This implementation results in a multiplier-less implementation, that provides a high performance system, as no multiplication is required. In [13], a modified version of the LMS algorithm (delayed LMS) is implemented on a Virtex-II FPGA with fully pipelined architecture to provide a high throughput. In this paper an architecture to implement MB RISC processor and HW accelerator is proposed. The accelerator is then used to build a pure HW implementation.

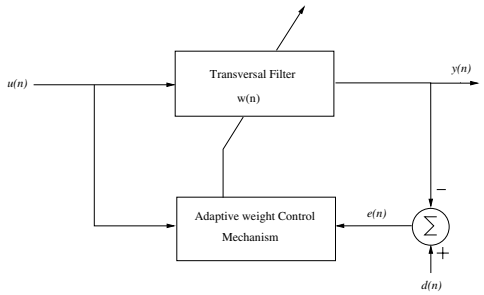


Figure 1. Simplified Block Diagram of LMS adaptive filter

A normal FIR filter based on MAC operations could be used to implement this algorithm. A weight update mechanism should be added to the FIR filter to update the filter weights according to the calculated error. This module requires two extra multiplications and a single addition.

In this paper the LMS algorithm is used for audio processing. The filter is trained to produce the desired output for a given audio signal. The implementation of this algorithm for audio processing requires three steps: (1) audio capture, (2) audio processing, (3) audio playback. The Xilinx multimedia board shown in Figure 2, is used for final implementation. The on-board AC97 codec is used for audio capture/playback and the FPGA chip is used to implement the three architectures (to be introduced in the following sections). The board provides a complete platform to implement multimedia applications based on Xilinx FPGAs. The board is mounted with audio ports, and controllers that are interfaced to the FPGA to enable transferring data directly to the chip. The board also contains a serial port connected to the FPGA for communication with other systems. The serial port is used for communication between the board and the PC, and to display user input/output [2].

Xilinx EDK 7.1 is used to implement the first two MB based architectures [4], while Xilinx ISE 7.1 is used for implementing the pure hardware implementation. All cores and hardware modules are described in VHDL, synthesized with Xilinx Synthesis Tool (XST). Simulations are performed using Xilinx ISE Simulator [1], and Xilinx ChipScope is used for hardware debugging.

3 Implementation

The LMS algorithm introduced in the previous section is described using the flowchart shown in Figure 3. First the

[1] Mentor Graphics ModelSim is another alternative simulator available.

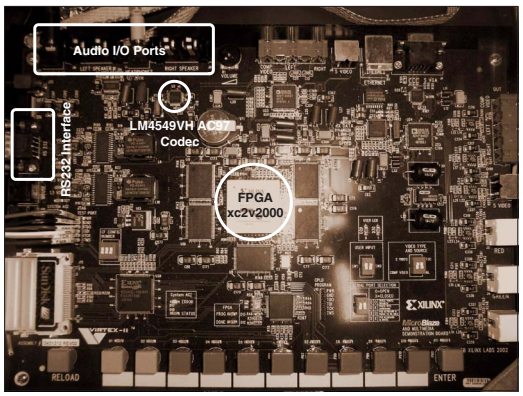


Figure 2. Multimedia Board

audio codec is initialized to start sound capture and playback. A sample is then captured from the audio codec. The filter output is computed for the captured sample. The error is computed and convergence is checked (if not reached the filter weights are updated). Next the filter output is played back using the audio codec. In the following subsections the three different architectures used to implement this algorithm are described.

3.1 Software Implementation

A block diagram of the first architecture is shown in Figure 4. The MB processor is used to run the pure software implementation of the algorithm. As shown the MB processor has three different buses, Local Memory Bus (LMB), On-Chip Peripheral Bus (OPB), and a Fast Simplex Link (FSL). The first bus is used to interface the MB with the instruction/data memory which in this system is a dual port block Ram. The OPB is used to interface the MB with different peripherals. In this system the MB is interfaced to the following OPB peripherals:

1. AC97 OPB CONTROLLER: is used to control the onboard audio codec [5]. It uses the OPB to initialize the codec, and uses FSL channels for audio capture/playback. The OPB could also be used for audio capture/playback, but the FSL is faster, since it uses only one instruction for data transfer.

2. OPB Timer: is used for profiling the software by counting the number of cycles required to complete a specific part of the program [9].

3. OPB RS232: is used for serial interfacing with the PC to transfer the user input/output data.

The FSL channels are used for audio data transfer from/to the AC97 controller. In the other two architectures

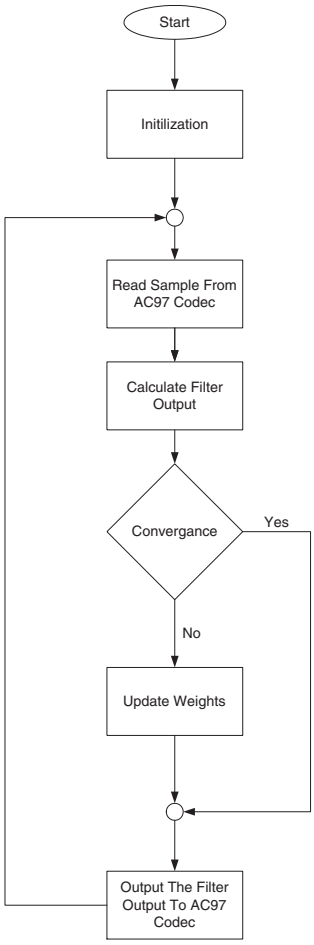


Figure 3. Flowchart of the Software Implementation

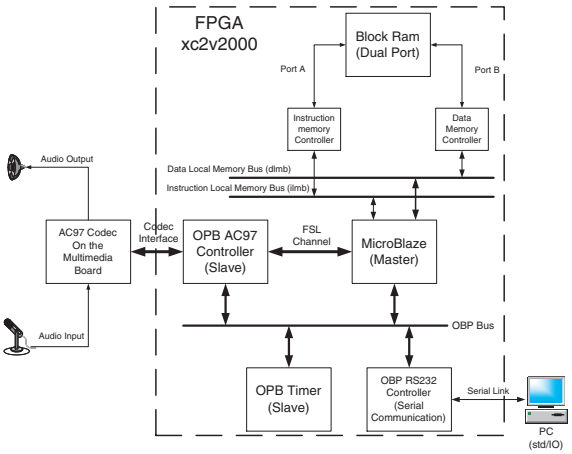


Figure 4. Pure Software System

Function	$N = 8$	$N = 16$	$N = 32$	# Iterations
AC97 Init.	13690	13690	13690	1
Filter Output	517	769	1473	Each Iter.
Convergence	660	660	660	Each Iter.
Weight Update	335	627	1171	Each Iter.

Table 1. Profiling Results of The LMS algorithm (Clock Cycles)

it is used to transfer data to/from the FIR core (i.e., used for acceleration).

The algorithm is written in C and profiled using the OPB timer. The timer is started before each operation and terminated when the operation is complete. The timer count represents the number of cycles required to complete this operation. The four main operations in the system are:

1. Initializing the AC97 codec.
2. Computing the filter output.
3. Error calculation and convergence checking.
4. Weight Update.

The profiling results of the four operations are shown in Table 1 for three different values of N. Results in 1 clearly indicate that the AC97 initialization process is a time consuming operation since many control words are transferred to the AC97 controller to specify the sampling rate, the input source, the input volume and the output volume. This operation is executed only once and thus is independent of N. The error calculation and convergence checking are executed each iteration, but are independent of N and do not affect the filtering operation. The remaining operations, filter output calculation and weights update are filter dependent and increase linearly with N. The pseudo code of the two functions is shown in Figure 5. The last two operations are selected to be implemented in hardware.

3.2 Software/Hardware Implementation

The second architecture proposed is based on a Co-design approach. As shown in the previous section, profiling the algorithm shows that the CalculateOutput and WeightUpdate operations could be moved to hardware. In this architecture a tap weights updatable FIR filter core is implemented in VHDL to replace and accelerate the two above mentioned operations. A block diagram of this architecture is shown in Figure 6. As shown the FIR core is connected to the MB processor using two FSL channels. The

```
d : int16 array length N; ## TAP Inputs
w : int16 array length N; ## TAP Weights
function CalculateOutput (int16 input) returns int32
    for i = 1 to N-1
      ## Move the TAP input one step
      d(i) = d(i-1);
    end;
    d(0) = input;
    for i = 0 to N-1
      ## Multiply Accumulate to get the output
      output = output + d(i)*w(i);
    end;]
    return output;
end CalculateOutput;

function WeightUpdate (int16 error_rate_prod)
## The input is the error and the learning rate product
    for i = 1 to N-1
      ## Move the TAP input one step
      w(i) = w(i)+ (d(i) * error_rate_prod);
    end;
    return;
end WeightUpdate;
```

Figure 5. Pseudo Code of CalculateOutput/WeightUpdate functions

first channel is used for data I/O from/to the filter. The second is used to send weights update data (Error-Rate Product) and to receive a confirmation of weight update completion. The remainder of the system is identical to the first architecture. The CalculateOutput is replaced with two FSL write/read operations to send the audio sample to the filter and read back the filter output. The error calculation, and convergence checking remain unchanged. If the weights need to be updated, the WeightsUpdate function is also replaced with two FSL write/read operations to send the error data to the filter and read back a confirmation.

The details of the FIR filter core are shown in Figure 7. Figure 7(a) shows a simple block diagram of the core. The core contains two FSL channel interfacing logic modules responsible for data transfer from/to the filter core. The first interfacing logic block reads data from the FSL channel, and transfers it to the LMS filter as shown in Figure 7(b). The filter consists of N tap unit as shown in Figure 7(c). Each tap contains two registers, the first holds the tap input while the other holds the tap weight. With the positive edge of the clock the tap unit latches its two inputs, multiplies them with a signed embedded multiplier. All the numbers are 16 bit signed numbers with the decimal point at position 15. The truncation module in the tap unit

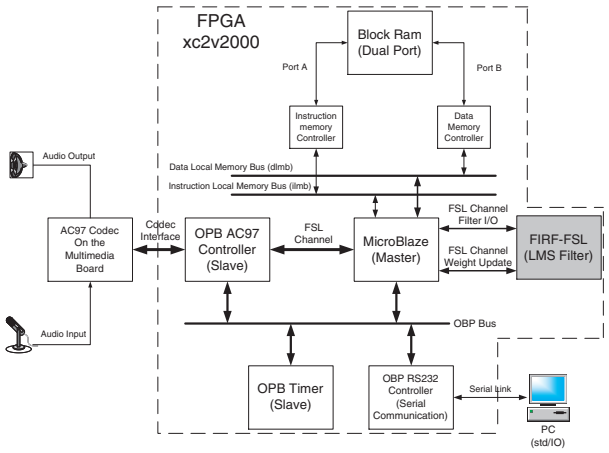


Figure 6. Hardware/Software Co-design

simply shifts the 32 bits multiplication result 15 bits to the right. With the negative edge of the clock, the tap units release its output, and sends its tap input to the next stage. The output of all the tap units inside the LMS filter is then added using an adder tree that produces the filter output. The FSL channel logic gives the LMS filter 4 clock cycles to compute its output and then starts transferring the results.

The second FSL interfacing logic is used to read the weights update data. When the FSL interfacing logic block reads the weights update data from the FSL channel it initiates a weights update process which requires N clock cycles. It uses a single multiplier and a single adder to update a single weight each clock cycle. Since the weight update process runs only when convergence is not reached, a single multiplier/adder is used to implement it. The profiling of the HW/SW architecture is shown in Table 2. The FIR filter core reduces the number of cycles required for both functions by a ratio close to 90%. It is also clear that the number of cycles required for the CalculateOutput is fixed, and is independent of N. As it is implemented to be computed in parallel in 4 clock cycles, the extra cycles are required by the MB processor to execute the function call, and perform the FSL read/write operations. For the WeightUpdate function, the number of cycles increases with N, one cycle for each extra tap.

3.3 Hardware Implementation

The third architecture is a pure hardware implementation of the algorithm and is shown in Figure 8 . This architecture makes use of the same FIR filter core used with HW/SW architecture. It also makes use of the AC97 controller core for controlling the audio codec. An extra two cores are added to replace the MB system. The first core is the AC97 initialization unit that interface with the AC97 controller

Function	$N = 8$	$N = 16$	$N = 32$	# Iterations
AC97 Init.	13690	13690	13690	1
Filter Output	87	87	87	Each Iter.
Convergence	660	660	660	Each Iter.
Weight Update	92	100	116	Each Iter.

Table 2. Profiling Results of The LMS algorithm after using HW accelerator(Clock Cycles)

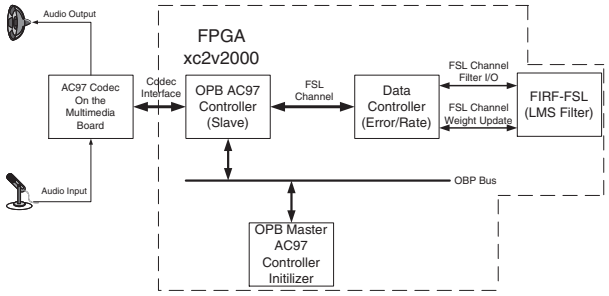


Figure 8. Hardware Implementaion

through the OPB bus for initialization. The initialization process requires writing a sequence of values into the AC97 controller register. The write operation to the OPB requires 3 clock cycles, while the read operation requires 4 clock cycles. As the initialization of the AC97 codec requires 11 register write operations, each requiring two OPB writes (Address, Data), and one OPB read (Status Reading), the total number of cycles required to initialize the codec is close to 108 clock cycles given that the codec is ready [5, 1].

The second core used is the data controller (error and rate calculation unit). It is a simple adder and comparator unit that performs its operation in 4 clock cycles and is responsible for the three following tasks:

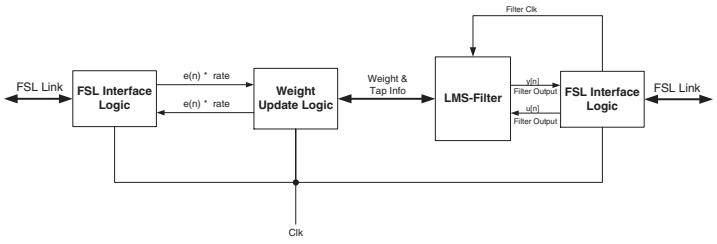
1. Control the communication between the FIR filter and the AC97 Controller core through FSL channels.

2. Calculate the filter error from the desired response and the filter output.

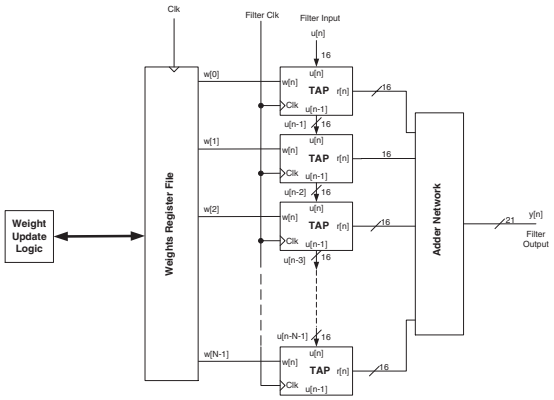
3. Check convergence and update weights if required.

As there is no MB involved in this architecture, the FSL access requires only one cycle. Consequently , the CalculateOutput will take only 4 cycles, and WeightUpdate will take N cycles.

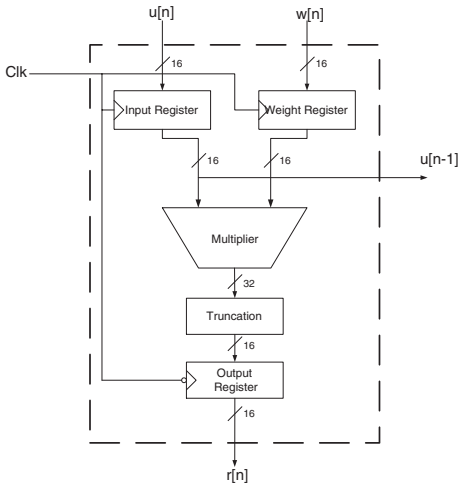
ChipScope is a tool that could be attached to hardware modules for on-chip data capture. It is used for hardware debugging. ChipScope is used to debug the audio signal capturing from the AC97 codec.



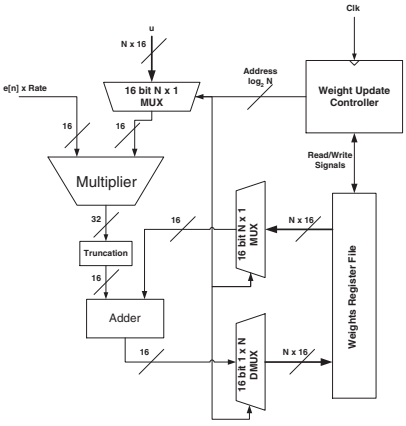
(a) FSL LMS FIR-Filter



(b) LMS Filter



(c) Single Tap



(d) Weights Update Logic

Figure 7. FSL LMS FIR-Filter Architecture

4 Results

In the previous section, three different architectures were proposed for realizing an LMS adaptive filter. The three architectures are implemented on the Xilinx multimedia board to capture an audio signal, process it and play it back. The desired response is chosen to provide a reduction in the audio level to reduce the noise in the audio signal. Convergence is checked each sample and once reached the weights update is no longer performed.

The profiling results for N equal to 32, for each architecture are shown in Figure 9. The comparison shows a significant improvement for the HW architecture over the other two. Figure 10 shows the speedup achieved for each architecture for different values of N. It is clear from Figure 10 that the pure HW implementation results in a speed up close to $82.6\times$ over the pure SW implementation (32 taps) and speed up of $20.1\times$ over the HW/SW implementation. The HW/SW implementation gives a speed up of $3.8\times$ over the SW implementation. The main clock source on the multimedia board is 27MHz, which is used for the three architectures.

The FPGA implementation results of all the cores used to implement the three architectures are shown in Table 3. The FIR core consumes a considerable amount of resources compared to the other cores. Its size linearly increases with N, and its operating frequency is the lowest compared to other units. Optimizing the adder tree used in the filter could result in a significant improvement of the resource requirements of the core. The FIR core greatly affects the overall FPGA implementation results shown in Table 4 for the three architectures. Table 4 and Figure 11 show

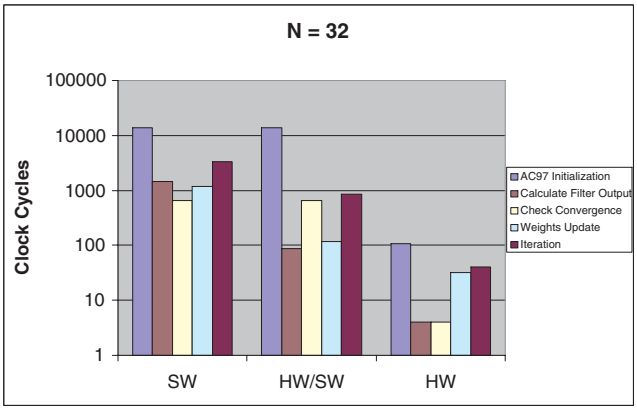


Figure 9. Profiling results for the 3 architectures for 32 taps

the FPGA resource utilization of each architecture. The maximum clock frequency for each architecture is shown in Figure 12. The SW implementation resource requirements are fixed since the algorithm is implemented on a single soft core, and changing N requires just modifying the C code and recompiling it. In the HW/SW implementation, the FIR core adds an extra resource to the SW system, and thus the area of the system is affected by N. In addition modifying N requires rebuilding the HW system. The clock frequency of the system is decreased due to adding the FIR core that performs the filter calculation. The HW implementation requires more resources with large values of N since the FIR core size increases with N. The clock frequency for the HW architecture is almost fixed and close to that of the SW architecture. It is also compared to the clock frequency for the FIR core as shown in Table 3. The HW architecture is designed to reuse the existing cores used with the HW/SW architecture.

Arch.	AC97	FIR Filter			AC97	Error
Length	Control	8	16	32	Initi.	Control
Slices (10752)	181	542	1073	2124	127	36
LUT4 (21504)	188	524	1032	2023	199	63
FF (21504)	137	682	1337	2649	76	55
MUL18 (56)	0	9	17	33	0	0
Freq.(MHz)	146	66	67	61	97	96

Table 3. Implementation Requirement For Each Core

Arch.	SW	SW/HW			HW		
Length	All	8	16	32	8	16	32
Slices(10752)	1173	1929	2232	3277	889	1395	2395
LUT4(21504)	1409	1700	2444	3432	793	1262	2172
FF(21504)	957	1612	2267	3579	975	1630	2942
MUL18(56)	3	12	20	36	9	17	33
Block-Ram(56)	32	32	32	32	0	0	0
EXT IO(624)	14	11	11	11	8	8	8

Table 4. Implementation Requirement for different architectures

Each of the three architecture is tested using a noisy speech sample from an external source. Real time filtering and adaptivity is verified for each architecture. Results obtained show a significant improvement in the sound quality.

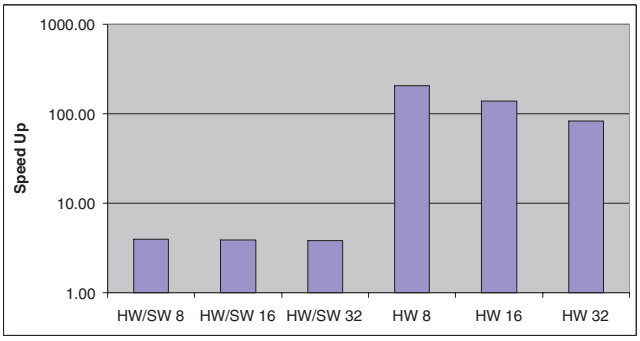


Figure 10. Speedup for 32 taps

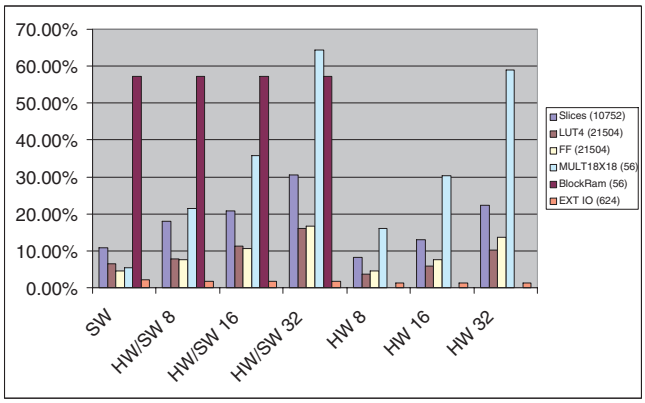


Figure 11. Resource Utilization for different architectures

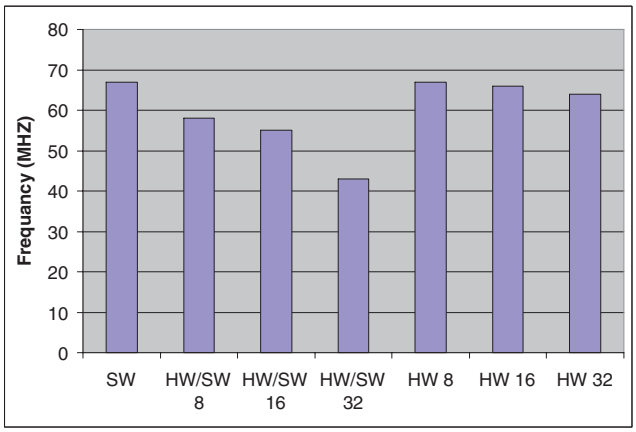


Figure 12. Maximum Frequencies for different architectures

5 Conclusion

In this paper three different architectures were proposed to implement an LMS adaptive filtering algorithm. The three architectures are aimed for audio processing using the Xilinx multimedia board and the MB soft core. A comparison between the three architectures shows that using a HW accelerator coupled with an MB processor in a Co-design configuration reduces the number of cycles required to perform the most two critical operations by about 90% with a total speedup of $3.8\times$. This improvement comes at a cost of extra area and lower level of flexibility. Using a pure HW architecture results in a speedup of $82.6\times$ with a moderated area, and lower flexibility.

References

[1] National semiconductors, lm4549 ac 97 rev 2.1 codec with sample rate conversion and national 3d sound data sheet, 2000.

[2] Xilinx inc., microblaze and multimedia development board user guide, 2002.

[3] Xilinx inc., virtex-ii platform fpga user guide, 2002.

[4] Xilinx inc., edk 7.1 user guid, 2006.

[5] Xilinx inc., ml40x edk processor reference design user guide for edk 8.1 -ac97 obp controller core, 2006.

[6] D. J. Allred, W. Huang, V. Krishnan, H. Yoo, and D. V. Anderson. An fpga implementation for a high throughput adaptive filter using distributed arithmetic. In *Proceedings of the 12th Annual IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM04)*, pages 324 – 325. IEEE, April 2004.

[7] U. M. Baese. *Digital Signal Processing with Field Programmable Gate Arrays*. Springer-Verlag, 2nd edition, 2004.

[8] S. Haykin. *Adaptive Filter Theory*. Pearson Education, 4th edition, 2002.

[9] X. Li and S. Areibi. A hardware/software co-design approach for face recognition. In *In 16th International Conference on Microelectronics, Tunis, Tunisia*, pages 67–70, Dec 2004.

[10] A. D. Stefano, A. Scaglione, and C. Giaconia. Efficient fpga implementation of an adaptive noise canceller. In *Proceedings of Seventh International Workshop on Computer Architecture for Machine Perception, 2005 (CAMP 05)*, pages 87–89. IEEE, July 2005.

[11] B. Widrow and S. D. Stearns. *Adaptive Signal Processing*. Prentice-Hall, 1985.

[12] M. E. Windrow B. Adaptive switching circuits. *IRE WESCON Conv. Rec.*, pages 96–104, 1960.

[13] Y. Yi, R. Woods, L. K. Ting, and C. F. N. Cowan. High speed fpga-based implementations of delayed-lms filters. *J. VLSI Signal Process. Syst.*, 39(1-2):113–131, 2005.